

Mi primera app Android

Jorge Moreno Rivero

Índice

Mi primera app Android **1**
Introducción **3**
Parte A · Análisis del desarrollo móvil **3**
A.1 Ventajas e inconvenientes del desarrollo móvil 3
A.2 Elección tecnológica 4
A.3 Ciclo de vida de una Activity 6
Parte B · Especificación de la app **7**
B.1 Idea y público objetivo 7
B.2 Objetivos medibles 8
B.3 Requisitos funcionales 8
B.4 Requisitos no funcionales 9
B.5 Casos de uso 10
Parte C · Entorno de desarrollo **11**
C.1 Creación del proyecto 11
C.2 Emuladores y pruebas 13
C.3 Estructura del proyecto 18
Parte D · Recursos de la app **20**
D.1 Pantallas 20
D.2 Textos e idiomas 20
D.3 Imágenes 20
D.4 Colores 21
D.5 Estilos y tema 23
D.6 Dimensiones 23
D.7 Icono de la app 23
Conclusión personal **24**
Uso de IA **24**
Referencias **25**

Introducción

Esta memoria recoge la Práctica 1 del módulo de Programación Multimedia y Dispositivos Móviles. El objetivo era analizar el desarrollo móvil, especificar una app propia y dejar su proyecto de Android Studio preparado con todos los recursos bien organizados.

Mi app se llama Al Fallo y sirve para anotar entrenamientos de gimnasio de alta intensidad: pocas series, pero llevadas hasta el fallo muscular. Como la práctica no pide programar la funcionalidad, la especificación de la parte B describe la app completa, pero solo he implementado la interfaz de dos pantallas.

El proyecto está en un repositorio público de GitHub: https://github.com/JorgeMorenoRivero/PMDM_P1_JorgeMorenoRivero

Parte A · Análisis del desarrollo móvil

A.1 Ventajas e inconvenientes del desarrollo móvil

Lo mejor de programar para móvil es que el teléfono va siempre con el usuario. Eso da acceso a cosas que un ordenador no tiene, como la cámara, el GPS, los sensores o las notificaciones, y a un público enorme que ya tiene el dispositivo en la mano. Las tiendas de aplicaciones también ayudan a llegar a la gente y a cobrar. Frente a una web, además, la app funciona bien sin conexión y se siente más fluida.

A cambio hay que trabajar con menos recursos que en un ordenador y con más variedad de aparatos. Una web se abre en cualquier navegador sin instalar nada, y en escritorio casi nunca te preocupa la batería. En móvil sí, y estas son las limitaciones que más afectarían a Al Fallo:

Limitación	Cómo afectaría a mi app

Batería	Se entrena una hora o más con la pantalla encendida. La app no debe usar GPS ni trabajar en segundo plano sin necesidad.
Memoria	Un móvil básico tiene poca RAM y Android cierra apps para recuperarla. Por eso conviene usar imágenes pequeñas, vectores donde se pueda, y guardar lo escrito antes de que el sistema cierre la pantalla.
Conectividad	En muchos gimnasios no hay cobertura, sobre todo en sótanos. La app tiene que guardar todo en el propio móvil y no depender de internet.
Tamaño de pantalla	Se usa de pie, con una mano y entre serie y serie. Hacen falta botones grandes (mínimo 56 dp) y poca información en cada pantalla.
Fragmentación	Hay miles de modelos y varias versiones de Android. Por eso fijo la API 24 como mínima y pruebo en un móvil y en una tablet.

A.2 Elección tecnológica

	Nativo	**Híbrido**	**Multiplataforma**
Lenguajes	Java o Kotlin (Android), Swift (iOS)	HTML, CSS y JavaScript dentro de un contenedor	Dart, JavaScript/TypeScript o C#

Rendimiento	El mejor: el código corre directamente en el sistema	El más bajo: depende del navegador interno (WebView)	Bueno, algo por debajo del nativo
Acceso al hardware	Total y desde el primer día	Limitado, mediante plugins	Amplio, mediante librerías intermedias
Coste	Más alto si hay que hacer una app por sistema	Bajo: un solo equipo web	Medio: un código para ambos sistemas
Ejemplo	Android Studio con Java	Ionic con Cordova	Flutter o React Native

Voy a desarrollar Al Fallo en nativo para Android. La práctica pide Java y vistas XML, pero además es una app sencilla que debe abrirse rápido y responder al instante entre series, y el nativo es lo que mejor lo consigue. Tampoco necesito iOS ni un equipo grande, que son las dos razones habituales para elegir híbrido o multiplataforma. Con el nativo también uso directamente los recursos de Android (textos, estilos, idiomas) tal como se explican en los temas.

A.3 Ciclo de vida de una Activity

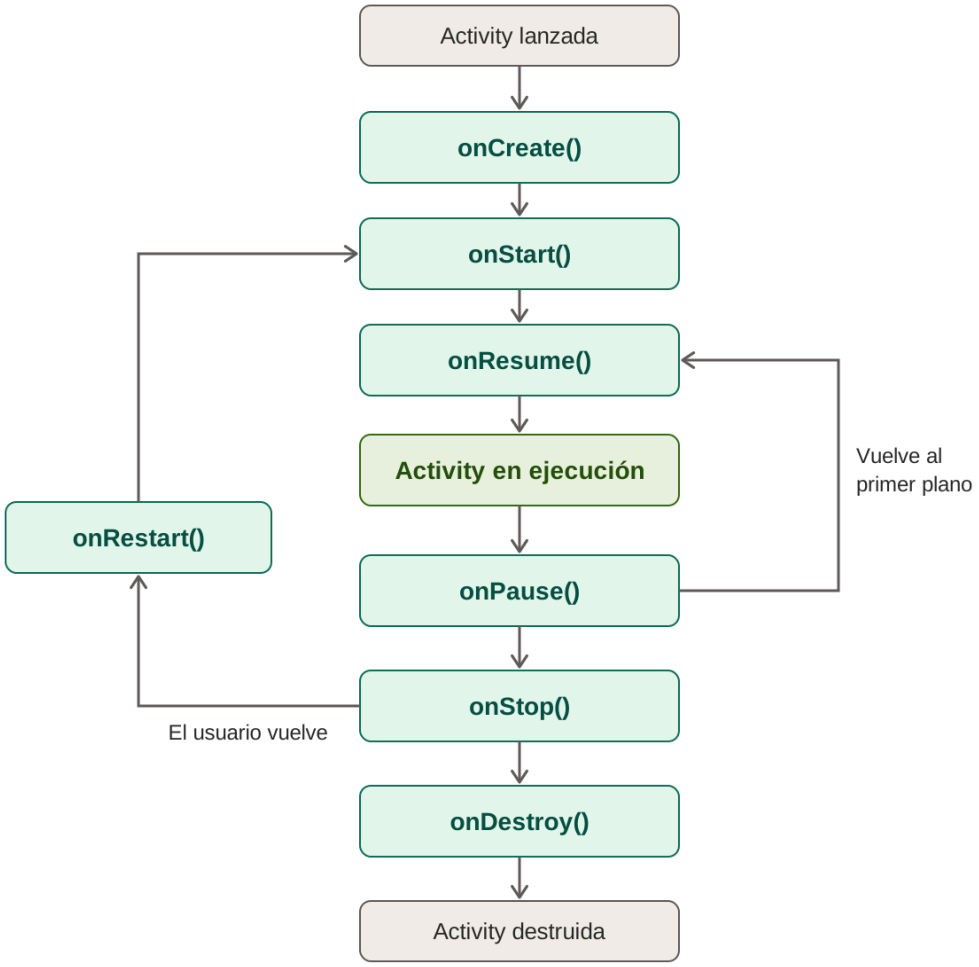


Figura 2. Ciclo de vida de una Activity

Una Activity pasa por varios estados y Android avisa de cada cambio llamando a un método. Estos son los tres casos que pide el enunciado, aplicados a Al Fallo:

El usuario recibe una llamada. Si la llamada ocupa toda la pantalla, la pantalla de registro pasa por onPause() y después por onStop(), porque deja de verse. Si solo sale un aviso encima, se queda en onPause(). En ese momento la app debería guardar el peso y las repeticiones que el usuario ya había escrito, por si Android la cierra mientras habla. Al

colgar, la Activity vuelve por onRestart(), onStart() y onResume() (o solo onResume() si nunca llegó a pararse) y el usuario sigue donde lo dejó.

El usuario gira el móvil. Android destruye la Activity y la vuelve a crear con la nueva orientación: onPause(), onStop(), onDestroy(), y luego onCreate(), onStart() y onResume(). Los campos de texto con id recuerdan lo que había escrito, pero las variables propias no. Si Al Fallo guardara, por ejemplo, el ejercicio elegido en una variable, habría que conservarlo en onSaveInstanceState() para no perderlo al girar.

El usuario pulsa Atrás. La Activity se cierra para siempre: onPause(), onStop() y onDestroy(). Si estaba en la pantalla de registro, el usuario vuelve a la de inicio, que sale de onStop() por onRestart(), onStart() y onResume(). Lo que no se haya guardado en ese momento se pierde, así que habría que guardar la serie antes de salir.

Parte B · Especificación de la app

B.1 Idea y público objetivo

Al Fallo es una app para apuntar lo que haces en el gimnasio sin complicarte la vida. Está pensada para gente que entrena con pocas series pero muy duras, llevando cada una hasta el fallo muscular. La mayoría de apps de entrenamiento están hechas para rutinas largas, con mil opciones, gráficos y planes de pago, y para quien hace tres series por ejercicio eso sobra. Aquí solo se elige el ejercicio, se anota peso y repeticiones, y se marca si se llegó al fallo. El público objetivo son personas de entre 18 y 35 años con algo de experiencia en el gimnasio, que entrenan varios días a la semana y quieren ver si progresan de una sesión a otra. Se usa de pie, entre serie y serie, con una mano y poco tiempo, así que todo tiene que ser rápido y legible.

B.2 Objetivos medibles

1. Que un usuario pueda registrar una serie completa (ejercicio, peso y repeticiones) en menos de 15 segundos.
2. Que la pantalla de inicio muestre el entreno del día con todos sus ejercicios sin necesidad de hacer scroll en un móvil estándar.
3. Que la app sea compatible con al menos el 99 % de los dispositivos Android activos (Android 7.0 en adelante, según el dato que da Android Studio al crear el proyecto).

B.3 Requisitos funcionales

Código	Requisito	Prioridad
RF-01	La app muestra el entreno del día y la lista de sus ejercicios	Alta
RF-02	El usuario puede registrar una serie indicando ejercicio, peso (kg) y repeticiones	Alta
RF-03	El usuario puede marcar si la serie se hizo hasta el fallo	Alta
RF-04	Al guardar una serie, la app muestra un mensaje de confirmación	Media
RF-05	El usuario puede consultar el historial de series de un ejercicio	Media
RF-06	La app se muestra en español o en inglés según el idioma del sistema	Media

RF-07	El usuario puede editar o borrar una serie ya guardada	Baja

B.4 Requisitos no funcionales

Código	Tipo	Requisito verificable
RNF-01	Rendimiento	La pantalla principal se abre en menos de 2 segundos en el emulador (se mide con cronómetro)
RNF-02	Usabilidad	Registrar una serie lleva como máximo 4 toques desde la pantalla de inicio
RNF-03	Usabilidad	Los botones miden al menos 56 dp de alto y los textos al menos 16 sp, para pulsarlos con el móvil en una mano
RNF-04	Compatibilidad	Android 7.0 (API 24) como versión mínima, porque cubre alrededor del 99,2 % de los dispositivos y permite usar Material 3 sin problemas
RNF-05	Compatibilidad	La interfaz se ve completa, sin cortes, tanto en el móvil como en la tablet del emulador
RNF-06	Mantenimiento	Ningún texto escrito a mano en los layouts: el Lint no muestra avisos de “Hardcoded string”

B.5 Casos de uso

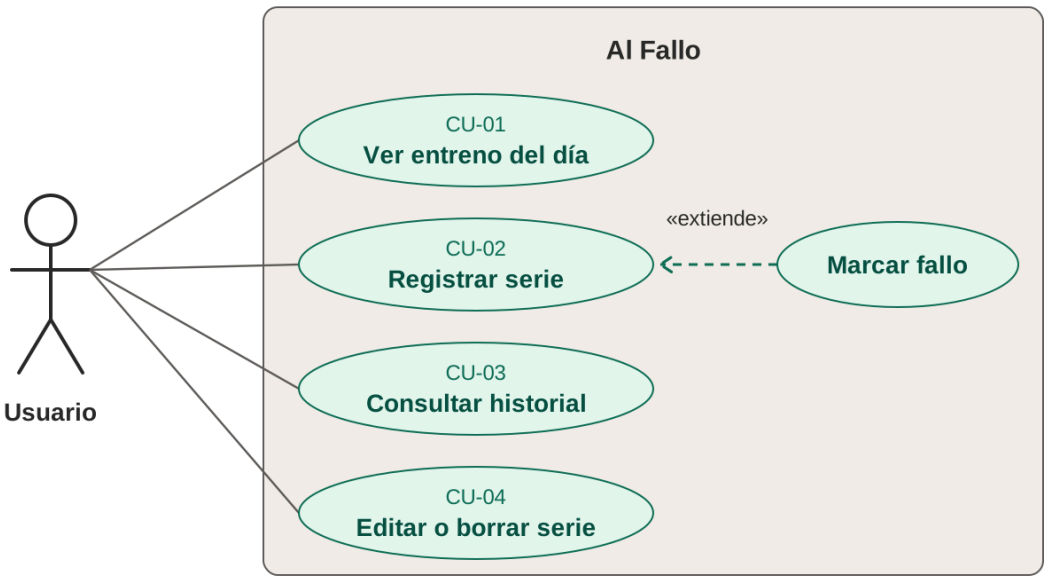


Figura 1. Diagrama de casos de uso de Al Fallo

Ficha CU-01 · Ver entreno del día

Actor: Usuario.

Precondición: la app está instalada y abierta.

Flujo principal: 1) El usuario abre la app. 2) La app muestra «Hoy toca: ...» con la lista de ejercicios. 3) El usuario pulsa «Empezar entreno».

Flujo alternativo: en el paso 2, si hoy no hay entreno planificado, la app muestra un mensaje de día de descanso y no ofrece el botón de empezar.

Postcondición: el usuario llega a la pantalla de registro con los ejercicios del día disponibles.

Ficha CU-02 · Registrar serie

Actor: Usuario.

Precondición: el usuario ha pulsado «Empezar entreno» y está en la pantalla de registro.

Flujo principal:

1. Elige un ejercicio de la lista.
2. Escribe el peso en kg.
3. Escribe las repeticiones.
4. Marca «Llegué al fallo» si es el caso.
5. Pulsa «Guardar serie».
6. La app guarda la serie y muestra un mensaje de confirmación.

Flujo alternativo: en el paso 5, si el peso o las repeticiones están vacíos, la app marca el campo y no guarda nada; el usuario lo corrige y vuelve a pulsar guardar.

Postcondición: la serie queda guardada con su fecha y hora y aparece en el historial del ejercicio.

Parte C · Entorno de desarrollo

C.1 Creación del proyecto

He creado el proyecto en Android Studio con la plantilla Empty Views Activity, lenguaje Java y API mínima 24. El nombre es PMDM_P1_JorgeMorenoRivero y el paquete es es.medac.jorgemoreno.app.

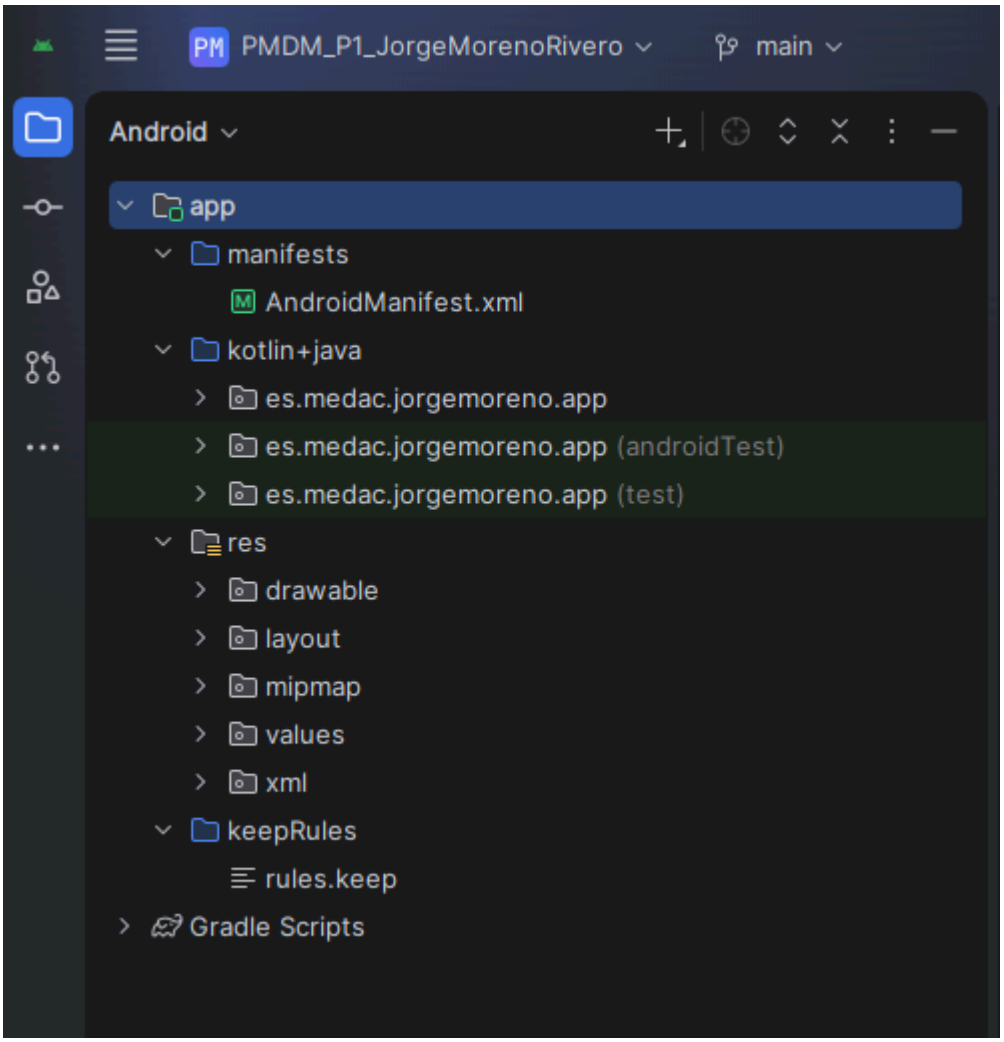


Figura 3. Proyecto creado en Android Studio

C.2 Emuladores y pruebas

Emulador	Resolución	Densidad	API
Medium Phone API 37.0	1080 × 2400 px (6,4")	420 dpi	37
Pixel Tablet	2560 × 1600 px (10,95")	320 dpi	37

Tabla 1. Emuladores usados

Para probar la app he creado dos emuladores con la misma imagen del sistema (Google Play, API 37): un móvil y una tablet. Así veo cómo se comporta la interfaz con dos tamaños de pantalla muy distintos. Los datos de cada uno están en la tabla anterior y en las figuras 4 y 5.

La app arranca y funciona bien en los dos. En el móvil la lista de ejercicios y el botón ocupan toda la altura de la pantalla. En la tablet, que es mucho más ancha, el contenido se estira a lo largo y la foto se recorta un poco por los lados gracias a centerCrop, pero no se corta nada de texto ni de botones.

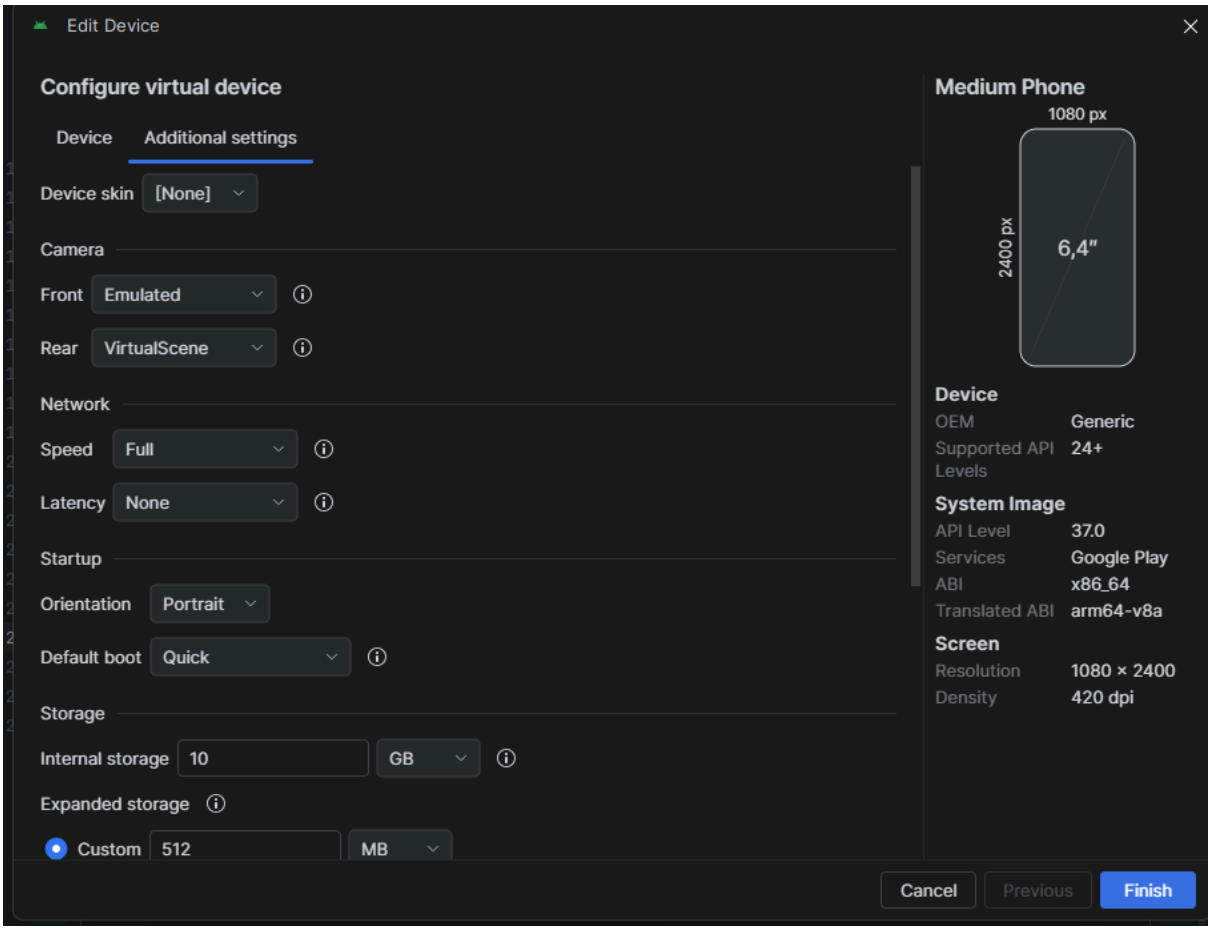


Figura 4. Datos del emulador de móvil en Device Manager

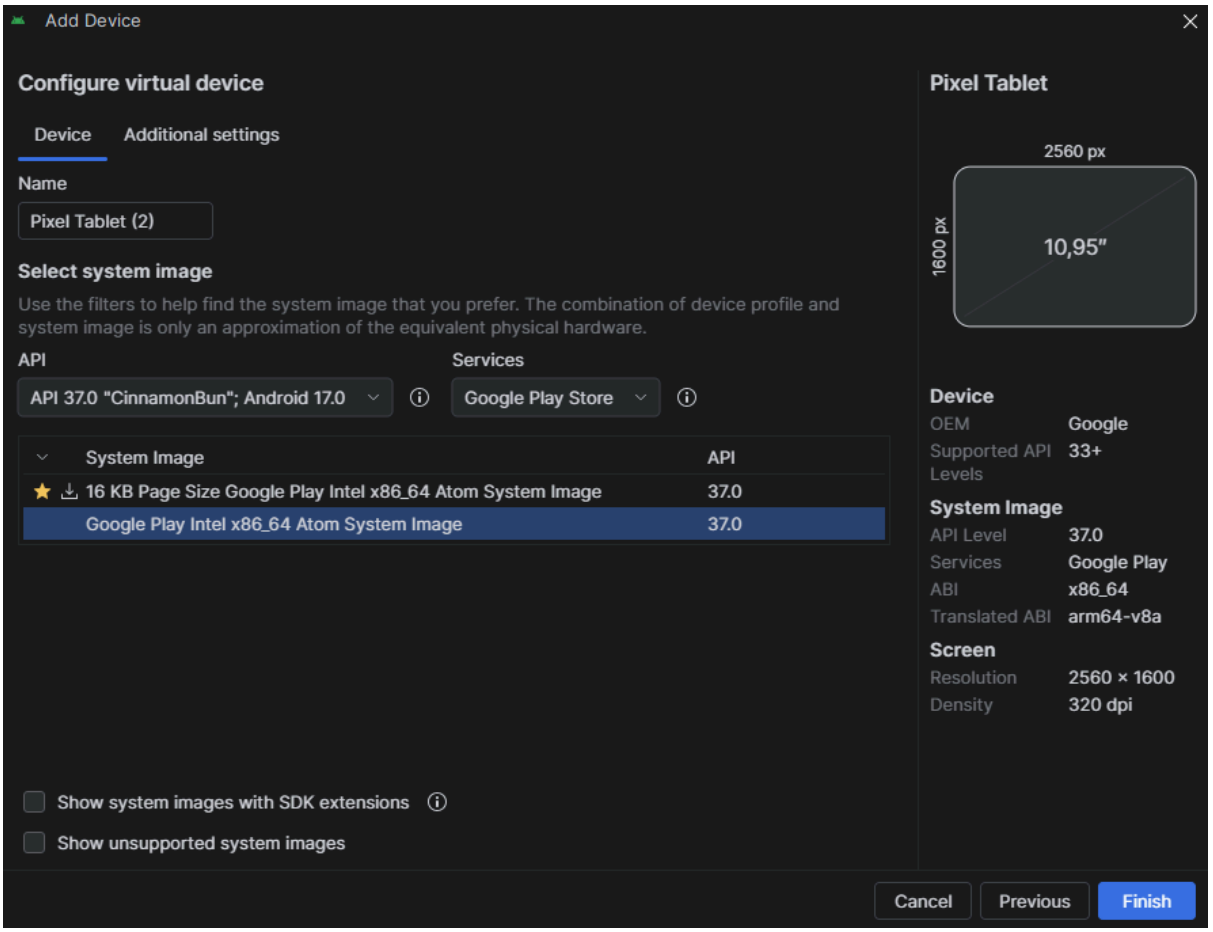


Figura 5. Datos del emulador de tablet en Device Manager

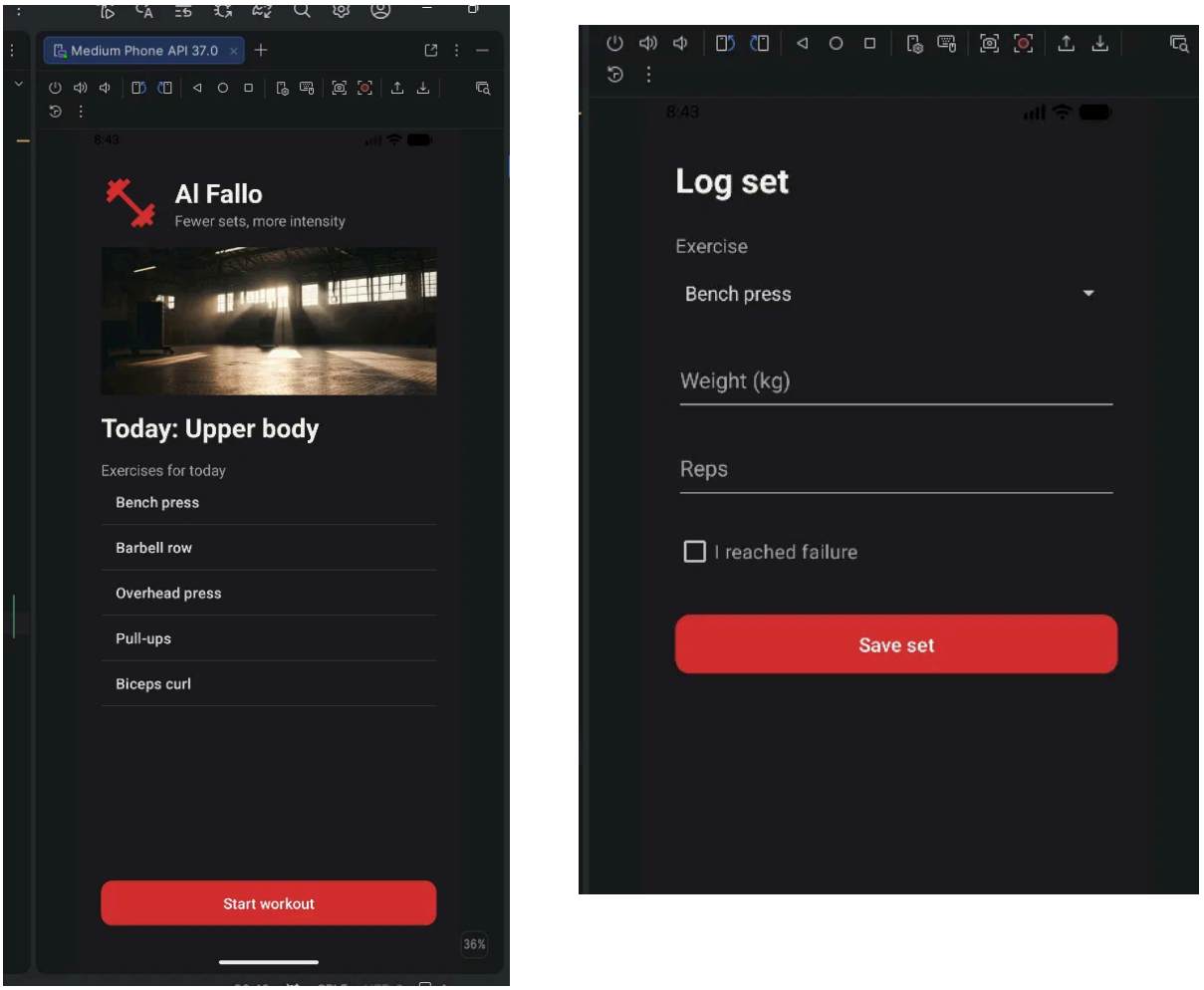


Figura 6. Al Fallo en el emulador de móvil

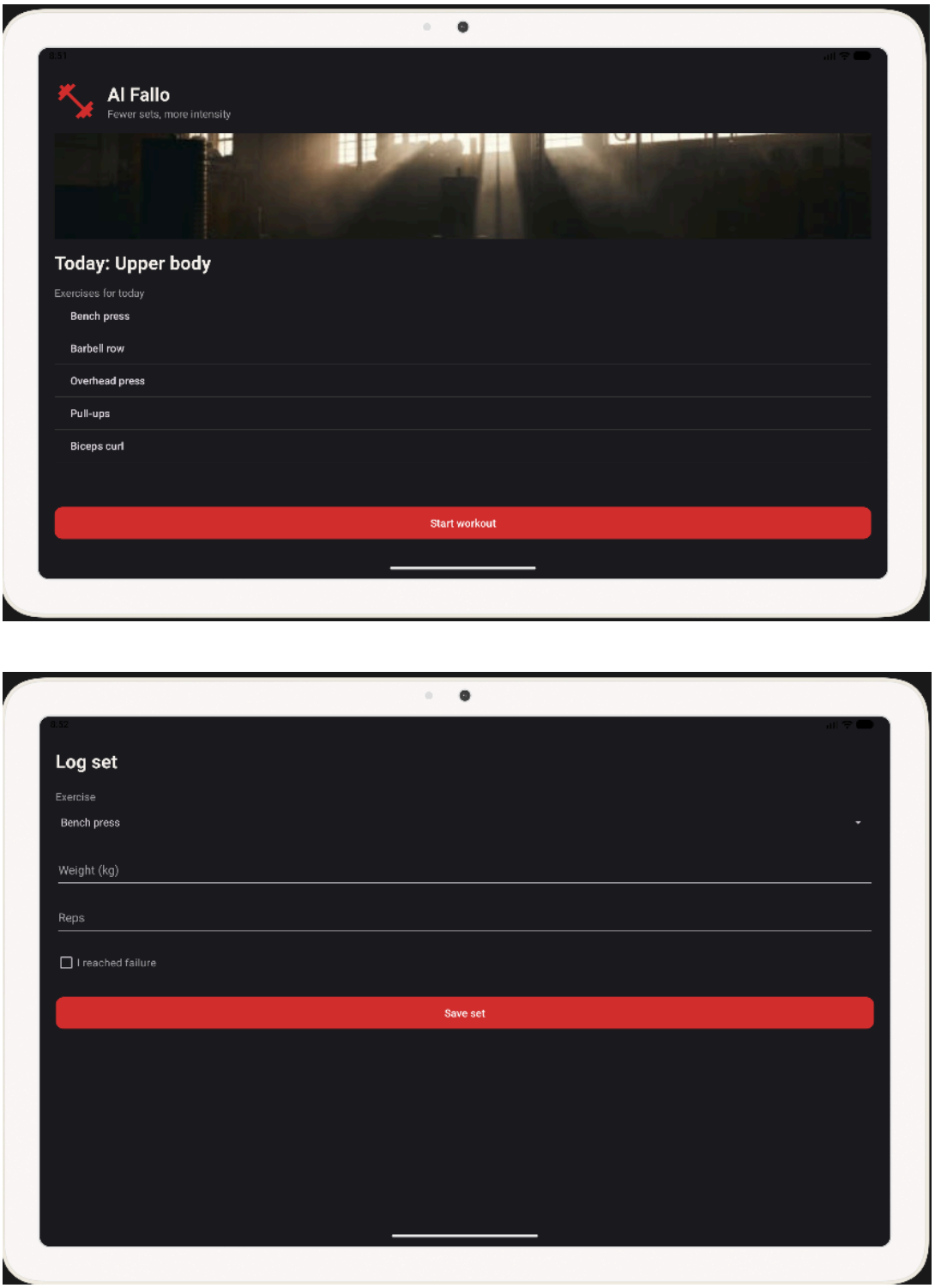


Figura 7. Al Fallo en el emulador de tablet

C.3 Estructura del proyecto

Al abrir el proyecto en la vista Android, las carpetas importantes son estas:

manifests: contiene AndroidManifest.xml. Ahí se declaran las dos Activities (MainActivity, que es la de inicio, y RegistroActivity), el icono y el tema de la app.

kotlin+java: el código. Está en el paquete es.medac.jorgemoreno.app y tiene MainActivity.java y RegistroActivity.java. Hay otras dos carpetas con el mismo paquete que son para tests y no las he tocado.

res/layout: los dos diseños en XML, activity_main.xml y activity_registro.xml.

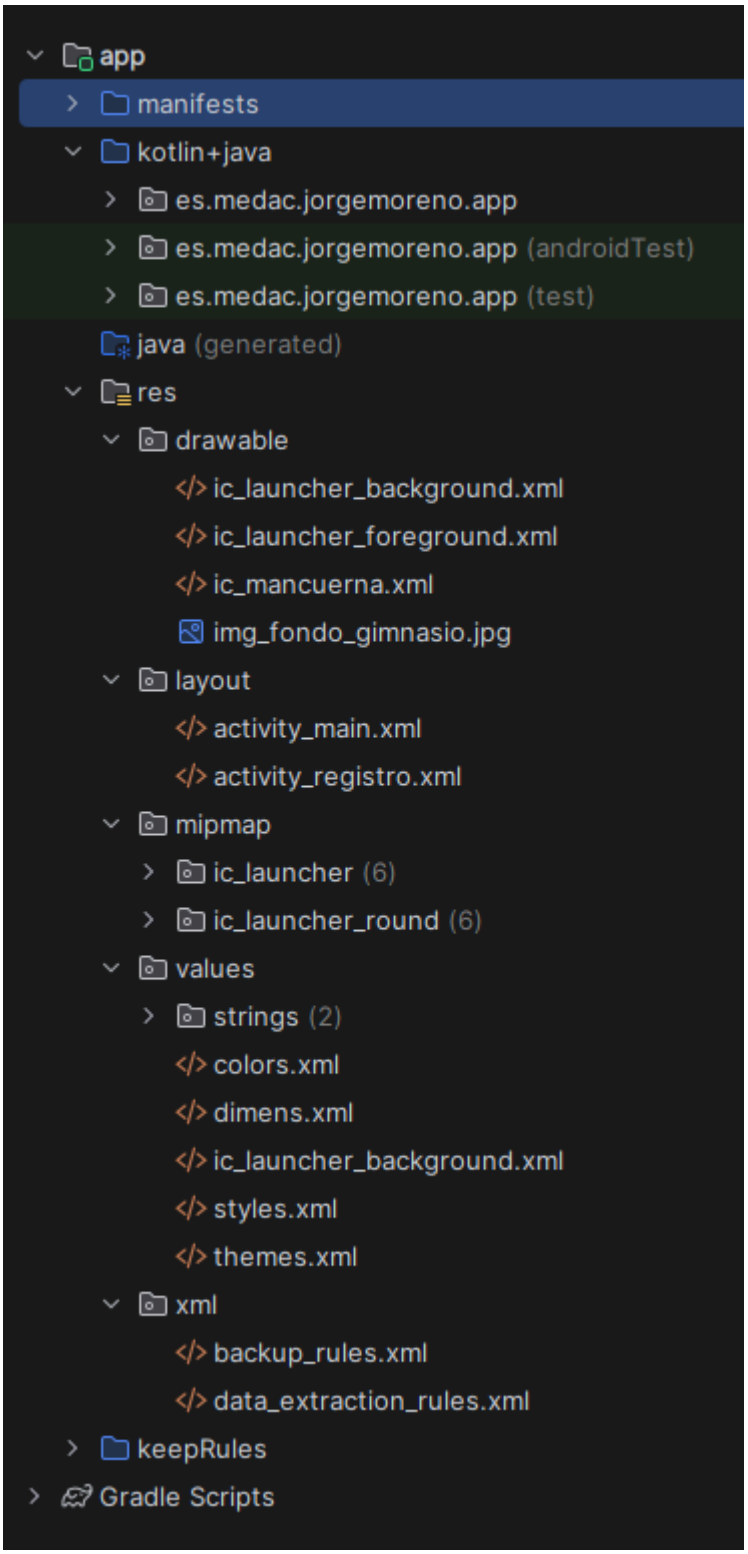
res/drawable: las imágenes: el vector de la mancuerna, la foto del gimnasio y los dos archivos del icono adaptativo.

res/mipmap: los iconos de la app en varias densidades, que genera Android Studio con Image Asset.

res/values: los recursos que no son imágenes. strings.xml tiene los textos, colors.xml la paleta, dimens.xml las medidas, styles.xml los estilos y themes.xml el tema. También está ic_launcher_background.xml con el color de fondo del icono.

res/values-en: la traducción al inglés de strings.xml. En la vista Android sale agrupada junto al original como «strings.xml (en)».

Gradle Scripts: los archivos de configuración de la compilación. Aquí está el minSdk 24, que es la versión mínima.


app
manifests
kotlin+java
es.medac.jorgemoreno.app
es.medac.jorgemoreno.app (androidTest)
es.medac.jorgemoreno.app (test)
java (generated)
res
drawable
ic_launcher_background.xml
ic_launcher_foreground.xml
ic_mancuerna.xml
img_fondo_gimnasio.jpg
layout
activity_main.xml
activity_registro.xml
mipmap
ic_launcher (6)
ic_launcher_round (6)
values
strings (2)
colors.xml
dimens.xml
ic_launcher_background.xml
styles.xml
themes.xml
xml
backup_rules.xml
data_extraction_rules.xml
keepRules
Gradle Scripts

Parte D · Recursos de la app

D.1 Pantallas

La app tiene dos pantallas. La de inicio (MainActivity) enseña el logo, el lema, una foto, el entreno de hoy con su lista de ejercicios y el botón «Empezar entreno». Ese botón abre la pantalla de registro (RegistroActivity), que tiene un desplegable para elegir ejercicio, dos campos para peso y repeticiones, una casilla para marcar si se llegó al fallo y el botón de guardar. Como la práctica no pide funcionalidad, el botón de guardar todavía no hace nada.

D.2 Textos e idiomas

Todos los textos están en res/values/strings.xml. Son 14 cadenas más una lista (string-array) con los cinco ejercicios del entreno de tren superior, que uso en la lista de la pantalla de inicio y en el desplegable de la de registro. Además, «Hoy toca: %1$s» lleva un parámetro, y desde MainActivity le paso el nombre del entreno con getString(R.string.entreno_hoy, getString(R.string.nombre_entreno)). Así el texto sale completo sin escribir nada a mano en el código.

La traducción al inglés está en res/values-en/strings.xml. No hace falta cambiar nada en la app: Android elige el idioma según la configuración del móvil. La versión en inglés es la de la Figura 6.

D.3 Imágenes

Uso dos imágenes de tipo distinto.

ic_mancuerna.xml es un Vector Asset (el icono fitness_center de Material, en rojo). Lo elegí vectorial porque es un dibujo sencillo, pesa muy poco y se ve nítido en cualquier tamaño. Lo uso como logo en la pantalla de inicio y como base del icono de la app.

img_fondo_gimnasio.jpg es una imagen de mapa de bits (612 × 408 px). Aquí no sirve un vector porque es una fotografía, con luces y sombras que no se pueden dibujar con trazos. La foto la saqué de una imagen que descargué de internet. La mantengo en un tamaño pequeño para no engordar la app.



Figura 8. Recursos de la carpeta drawable

D.4 Colores

Los colores están en colors.xml con nombre propio. Elegí un fondo gris muy oscuro con un rojo como color principal, porque da sensación de intensidad y de gimnasio, y con el fondo oscuro la pantalla cansa menos en sitios con poca luz.

Nombre	Valor	Uso y por qué
color_primario	#D32F2F	Rojo de la marca: botones, logo y casillas. Es

Nombre	Valor	Uso y por qué
		el color que llama la atención.
color_primario_oscuro	#9A0007	Variante más oscura del rojo para los contenedores del tema.
color_fondo	#1C1C1E	Fondo de las pantallas, un gris casi negro que resalta el rojo.
color_superficie	#2B2B2E	Fondo de elementos que van encima, un poco más claro que el fondo.
color_texto_principal	#F5F3EF	Títulos y textos importantes, blanco roto, más suave que el blanco puro.
color_texto_secundario	#A8A8AD	Subtítulos y pistas, gris para que pesen menos.
color_texto_sobre_primario	#FFFFFF	Texto de los botones rojos, blanco para que se lea bien.

D.5 Estilos y tema

He creado un tema propio, Theme.PMDM_P1_JorgeMorenoRivero, que hereda de Theme.Material3.Dark.NoActionBar. En él defino el rojo como color primario y el gris oscuro para fondo y superficies, con los textos claros. Todas las pantallas usan este tema desde el manifest.

En styles.xml hay tres estilos propios:

EstiloTitulo: texto de 28 sp, en negrita y con el color de texto principal.

EstiloTextoSecundario: texto de 16 sp con el color secundario.

EstiloBoton: hereda de Widget.Material3.Button, con 16 sp, texto blanco, fondo rojo, esquinas redondeadas de 12 dp y una altura mínima de 56 dp.

Así los dos botones y los títulos de las pantallas salen iguales sin repetir atributos en los layouts.

D.6 Dimensiones

Las medidas están en dimens.xml en dp (márgenes de 24 y 16 dp, logo de 64 dp, foto de 160 dp de alto y campos de 56 dp). Los tamaños de letra van en sp, que es la unidad que respeta si el usuario cambia el tamaño de fuente del sistema. Así el diseño es coherente y se cambia todo desde un solo archivo.

D.7 Icono de la app

El icono lo he creado con Image Asset de Android Studio como icono adaptativo. La capa de primer plano es mi mancuerna roja (ic_launcher_foreground.xml) y la capa de fondo es un color liso, #1C1C1E, definido en ic_launcher_background.xml. Android Studio genera los mipmap de cada densidad y la carpeta mipmap-anydpi-v26 para los iconos adaptativos. Se aplica en el manifest con android:icon y android:roundIcon.

Conclusión personal

Con esta práctica he aprendido sobre todo que una app Android es mucho más que escribir código: hay que organizar bien los recursos. Hacer los textos en strings.xml, los colores en colors.xml y las medidas en dimens.xml parece más trabajo al principio, pero luego cambiar algo es muy fácil, y la traducción al inglés fue casi inmediata gracias a eso.

Lo que más me costó fue la parte de preparar el entorno. Android Studio tarda bastante en abrirse y descargar, el emulador va lento la primera vez, y tuve algún despiste con Git: un commit no se subió a GitHub a la primera y lo vi al revisar el historial. También tuve que cambiar la ruta del proyecto porque no me dejaba guardarlo en C:\.

Me ha gustado ver mi propia app funcionando en el móvil y en la tablet con un icono hecho por mí. Me falta aprender a guardar los datos y a programar los botones, que es lo que viene en las próximas prácticas.

Uso de IA

Según permite la práctica, he usado Claude (Anthropic) como apoyo. Me ha servido de guía paso a paso para el entorno y el control de versiones, para proponer la temática y la paleta de la app, y para preparar borradores de los textos y de los diagramas, que después he

revisado y adaptado. El proyecto, los recursos y las capturas los he hecho yo en Android Studio.

Referencias

- Documentación oficial de Android para desarrolladores (developer.android.com): ciclo de vida de una Activity, recursos de la app y localización.
- Claude, de Anthropic (claude.ai), como herramienta de apoyo.